Design Document: PDF Upload & Query Application

This document provides an overview of the high-level and low-level architecture for the PDF Upload & Query application, detailing how the various components interact and operate.

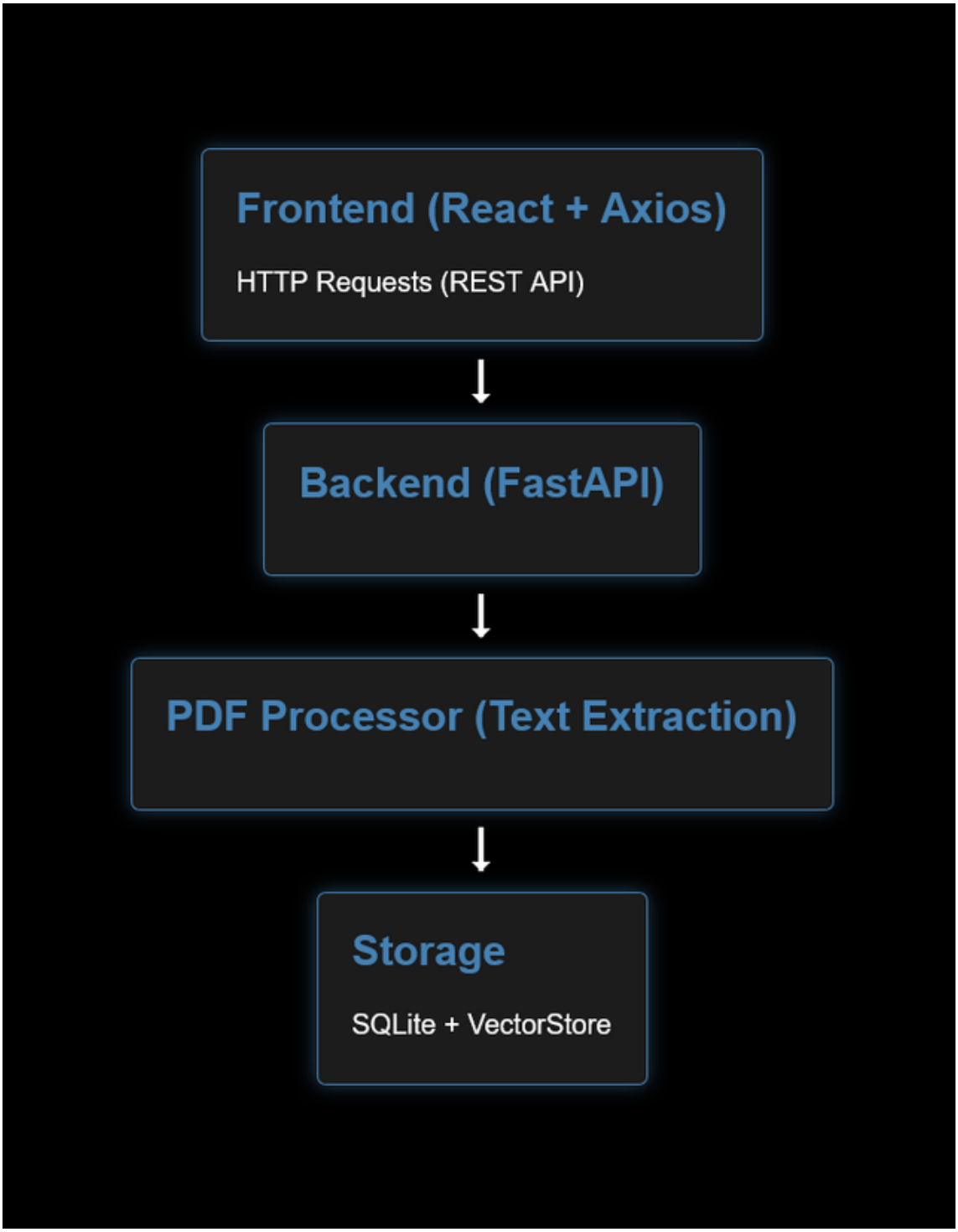
High-Level Design

Overview

The application is a full-stack system that allows users to upload PDF documents, retrieve metadata about these documents, query content within them, and manage (delete) stored PDF documents. The core technologies include FastAPI for the backend, SQLite for database storage, and React for the frontend.

Component Diagram

Below is a high-level component interaction diagram for the application:



System Flow

1. **Upload a PDF**: The frontend initiates a file upload. The backend processes the file, extracting text content and metadata, and storing it in both the SQLite database and a vector storage system.

2. **Retrieve Documents**: The frontend requests a list of uploaded documents. The backend retrieves this metadata from the SQLite database and responds with the data.
3. **Query PDF Content**: Users can send a query related to a specific document. The backend uses vector-based similarity search to return relevant document excerpts.
4. **Delete a PDF**: Users can delete a document from both the SQLite database and vector storage.

Low-Level Design

Backend Components

1. **FastAPI Server**:
   - Handles HTTP requests and routes.
   - Manages PDF uploads, document queries, and deletions.
2. **SQLite Database**:
   - Stores metadata about each PDF, including filename, hash, and upload date.
   - Ensures data persistence and provides a unique identifier for each document.
3. **Text Extraction**:
   - Uses PyMuPDF to extract text from uploaded PDFs.
   - Extracted text is stored in the database and indexed for efficient querying.
4. **Vector Store**:
   - Stores vectorized representations of each document's content for fast similarity search.
   - Each document is indexed using a unique folder within `vector_stores`.

Frontend Components

1. **UploadPDF Component**:
   - Allows users to upload PDF files.
   - Uses Axios to send files to the backend's `/upload-pdf/` endpoint.
2. **PDFList Component**:
   - Displays a list of all uploaded documents.
   - Allows users to select or delete documents, triggering additional API calls.
3. **AskQusetion Component**:
   - Provides a text input for users to query document content.
   - Displays query responses received from the backend.

Database Schema

The SQLite database has a single table named `documents`:

Column	Type	Description
id	INTEGER	Primary Key, unique identifier for each document.
filename	TEXT	Original name of the uploaded file.
content	TEXT	Extracted text content of the PDF (optional).
file_hash	TEXT	SHA-256 hash for duplicate detection.
uploaded_date	TIMESTAMP	Timestamp of when the PDF was uploaded.

Detailed API Workflow

Upload PDF Endpoint

1. **Endpoint**: `/upload-pdf/`
2. **Request**:
   - Accepts a file via multipart form-data.
3. **Process**:
   - Saves the file in the `/uploads` directory.
   - Extracts text from the PDF and computes a SHA-256 hash.
   - Checks for duplicate files by searching for the hash in the database.
   - Stores metadata and vector index for unique files.
4. **Response**:
   - Returns the document ID and filename if successful.
   - Returns a duplicate error if the file already exists.

Delete PDF Endpoint

1. **Endpoint**: `/delete-pdf/{document_id}`
2. **Request**:
   - Accepts a `document_id` in the URL.
3. **Process**:
   - Finds the document in the SQLite database by ID.
   - Deletes the associated file and vector index if they exist.
4. **Response**:
   - Returns a success message if deletion is successful.
   - Returns an error if the document doesn’t exist.

Get All Documents Endpoint

1. **Endpoint**: `/documents`
2. **Request**:
   - Simple GET request.
3. **Process**:
   - Queries the SQLite database for all documents, excluding content.
4. **Response**:
   - Returns a JSON array of document metadata.

Error Handling

1. **File Validation**: Ensures that only PDFs can be uploaded.

2. **Duplicate Handling**: Uses file hashes to prevent duplicate uploads.
3. **HTTP Exception Handling**: Returns appropriate HTTP status codes for missing documents, file errors, and server issues.